# An AI Tutor for Learning Support

## Built with Flask and Google Gemini

Submitted by: [Student Full Name] | Register No: [Register No.]

Guide: [Guide Name] | Department of Computer Applications

[College Name], [University Name] | Academic Year: [20XX - 20XX]

# Problem Statement and Objectives

## Problem

- Students need quick, simple help while studying, but searching many websites wastes time.
- Search results are scattered and often too advanced for beginners.

## Objectives

- Build a single web page that works as a personal AI tutor.
- Answer questions, explain topics, summarize text, create quizzes and suggest learning plans.
- Use Google Gemini for accurate, simple-English responses.
- Keep the app easy to install and run on a normal laptop.

# Features of EduGenie

### Ask a Question

Type any doubt and get a short, clear answer.

### Get an Explanation

Simple definition, explanation and example for any topic.

### Summarize Content

Paste long text and get 3 to 5 key points.

### Generate a Quiz

5 multiple-choice questions with answers on any topic.

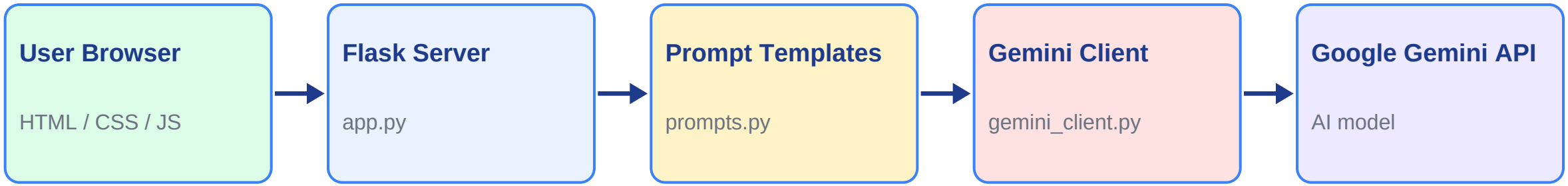
### Learning Recommendations

A step-by-step personalized plan for a topic.

# Technology Stack

| Technology | Purpose |
| --- | --- |
| Python 3 | Main programming language |
| Flask | Web server and REST API |
| HTML, CSS, JavaScript | Single-page user interface and fetch calls |
| Google Gemini API (google-genai) | Generates all AI responses |
| python-dotenv | Loads the API key from the .env file |
| unittest | Automated tests with a mocked AI call |
| Visual Studio Code | Development environment |

# System Architecture




The user types text in one of five sections. JavaScript sends it to Flask, Flask builds a prompt for that feature, the Gemini client calls the API, and the answer is shown in the result box.

# Working and API Endpoints

- User types an input and clicks a button (or presses Enter).
- JavaScript sends POST /api/<feature> with the text.
- Flask validates the input (not empty, max 5000 characters).
- The matching prompt template is filled and sent to Gemini.
- The reply is returned as JSON and shown in the result box.

| Method | Endpoint | Purpose |
|---|---|---|
| GET | / | Home page |
| GET | /health | Server status |
| POST | /api/<feature> | ask, explain, summarize, quiz, recommend |

# Project Structure and Modules

```
edugenie/
  app.py
  gemini_client.py
  prompts.py
  templates/index.html
  static/style.css
  static/script.js
  tests/test_app.py
  requirements.txt
  .env
```

- app.py: routes, validation, error handling
- gemini_client.py: calls Gemini, checks API key
- prompts.py: one template per feature
- index.html, style.css, script.js: user interface
- test_app.py: 10 automated tests

# Implementation Highlight

Core route in app.py: one endpoint serves all five features.

```
@app.post("/api/<feature>")
def run_feature(feature):
    cfg = FEATURES.get(feature)
    if cfg is None:
        return jsonify(error="Unknown feature."), 404
    text = (request.get_json(silent=True) or {}).get("text", "").strip()
    if not text:
        return jsonify(error="Please type something first."), 400
    try:
        result = gemini_client.generate_text(cfg["template"].format(text=text))
    except gemini_client.MissingKeyError as e:
        return jsonify(error=str(e)), 503
    except gemini_client.GeminiError as e:
        return jsonify(error=str(e)), 502
    return jsonify(label=cfg["label"], result=result)
```

# Output: Home Page and Ask a Question

**Input: "which is the largest ocean?"**

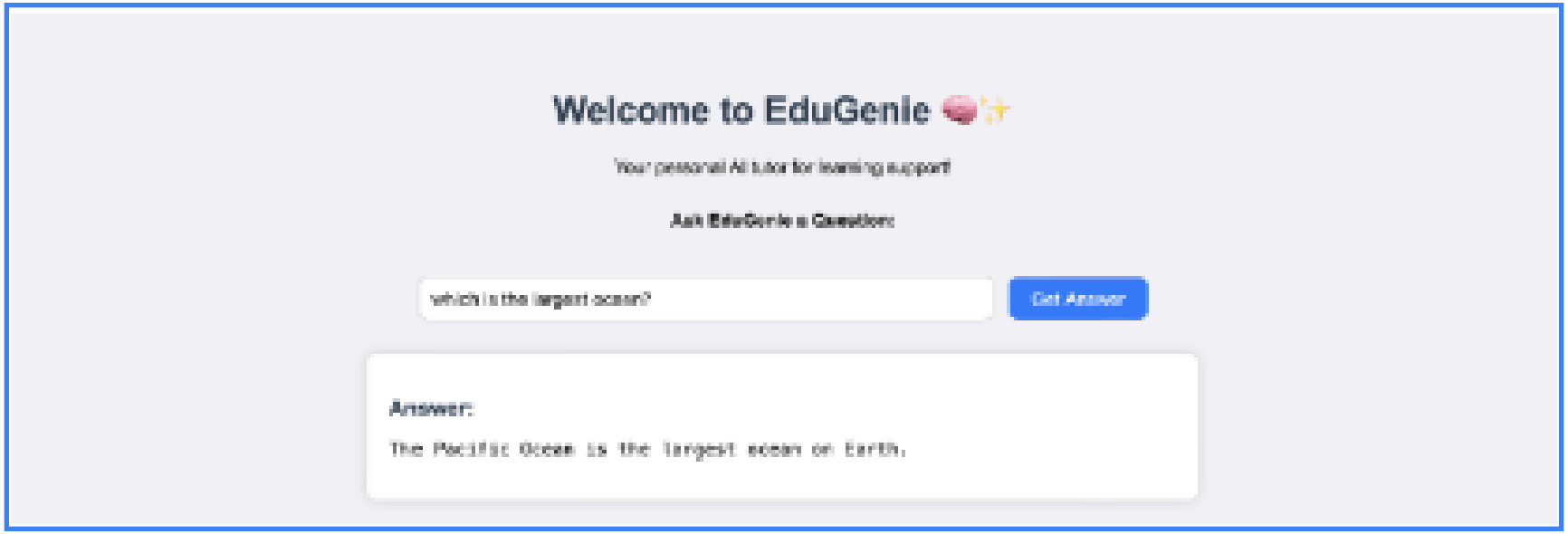



*Figure: Reference output from the EduGenie project documentation (replace with a screenshot from your own run).*

# Testing

| Test | Expected Result |
| --- | --- |
| Home page loads | Status 200, title shown |
| All 5 features (mocked AI) | Result returned |
| Empty or too-long input | Status 400 |
| Unknown feature | Status 404 |
| Missing API key | Status 503, clear message |
| Gemini failure | Status 502, error shown |

```
Run with: python -m unittest discover -s tests -v (10 tests)
```

# Conclusion and Future Scope

## Conclusion

- EduGenie gives students five learning tools on one simple page.
- Modular code makes prompts, API calls and UI easy to change.

## Future Scope

- User login and saved history
- Tamil and multi-language support
- Interactive quiz with scoring
- Upload PDFs and notes
- Deploy online with Docker or cloud hosting

# Thank You

Questions?